

1. How an object calls methods

When you write:

```
obj.method()
```

Python does NOT “call inside the object.”

Instead it does this:

Step 1: Find the function

It searches:

obj → class → parent classes (MRO)

Step 2: Bind the object automatically

Python converts:

```
obj.method()
```

into:

```
Class.method(obj)
```

Crux

That’s why `self` exists—it is the object passed manually in disguise.

When an object calls a method in Python, it does not directly execute the function inside the object. Instead, Python first searches for that method in the class of the object. If it does not find it there, it continues searching in parent classes according to the Method Resolution Order (MRO). Once the method is found, Python automatically passes the object itself as the first argument, which is called `self`. This is why `obj.method()` internally behaves like `Class.method(obj)`.

2. Runtime type vs reference type

Python does NOT behave like C++/Java.

So there is no strict “declared type.”

Instead:

- **Reference (variable)** → just a name pointing to object
- **Runtime type** → actual object in memory

`x = 10`

- `x` = reference (label)
- `10` = object (real thing in memory)

Python only cares about the object at runtime, not the variable.

Crux

The runtime type in Python refers to the actual type of the object that exists in memory while the program is running. The reference type refers to the variable that is pointing to that object. In Python, variables do not have fixed types; instead, they only store references to objects. Therefore, the actual type is always determined at runtime based on the object stored in memory, not the variable name.

3. What happens in memory when object is created

When you do:

`obj = A()`

Python does 3 hidden steps:

Step 1: Memory allocation

Object space is created in the heap.

Step 2: new runs

The object is physically created.

Step 3: init runs

The object is filled with data.

So:

obj → reference (name)

heap → actual object

Crux

When an object is created in Python, memory is first allocated in the heap for that object. After that, the constructor initializes the values of the object. The variable used to create the object does not store the object itself but stores a reference to the memory location where the object exists. Therefore, the object exists in heap memory while the variable acts as a pointer to it.

4. Why constructor is split into new and init

Because Python separates the following:

Creation (hardware-level idea)

→ `__new__` = “make object in memory”

Initialization (logical setup)

→ `__init__` = “fill values”

So conceptually:

Python first creates the body, then fills it with life.

Crux

The constructor in Python is split into `__new__` and `__init__` because object creation happens in two steps. The `__new__` method is responsible for creating and returning a new object in memory. After that, the `__init__` method initializes the object by assigning values to its attributes. This separation allows Python to first create the object and then configure it.

5. Can constructors be overridden or polymorphic?

Yes.

Because `__init__` is just a method.

So the child class can completely replace it:

Class A:

```
def __init__(self):  
    print("A")
```

class B(A):

```
def __init__(self):  
    print("B")
```

This is polymorphism: same name, different behavior.

Crux

Yes, constructors in Python can be overridden in child classes because `__init__` is a normal method. When a child class defines its own constructor, it replaces the parent class constructor. This allows different classes to initialize objects in different ways, which is a form of polymorphism. Constructors can also behave differently based on the number or type of arguments used.

6. Why constructor is not inherited

Technically it IS inherited, but:

Python does NOT force parent constructor execution.

Why?

Because the child class may need a different setup.

So inheritance means:

- “You may reuse it.”
- NOT “you must use it”

Crux

In Python, constructors are actually inherited, but they are not automatically used by child classes. This is because the child class may require different initialization logic. Therefore, Python allows the child class to override the constructor and define its own behavior. If needed, the parent constructor can still be called explicitly using `super()`.

7. Why calling overridden method in constructor is risky

Because the object is still under construction.

When the constructor runs:

child part may NOT exist yet

So if the constructor calls a method that the child overrides:

- child method runs too early

- attributes may not exist → error

So the problem is:

You are using a half-built object.

Crux

Calling an overridden method inside a constructor is risky because the object may not be fully initialized at that time. If the method being called depends on attributes that are not yet created in the child class, it can lead to errors or unexpected behavior. This happens because constructors run during object creation, when the object is still in an incomplete state.

8. Does Python support multiple inheritance?

Yes.

Because Python allows

```
class C(A, B):  
    pass
```

But it avoids chaos using the following:

MRO (Method Resolution Order)

So multiple inheritance is safe because Python has a strict search order system.

Crux

Yes, Python supports multiple inheritance, which means a class can inherit from more than one parent class. It supports this feature because Python uses Method Resolution Order (MRO) to manage conflicts between parent classes. MRO defines a clear and consistent order in which classes are searched, making multiple inheritance safe and predictable.

9. What is MRO

MRO is simply:

The exact order Python follows to search methods.

Example:

C(A, B)

Search order:

C → A → B → object

So Python NEVER guesses—it follows a fixed path.

Crux

Method Resolution Order is the order in which Python searches for a method when it is called on an object. If the method is not found in the current class, Python searches parent classes in a specific sequence. This sequence is determined by the C3 linearization algorithm, which ensures a consistent and conflict-free search path in inheritance.

10. What super() actually does

Most people misunderstand this.

`super()` does NOT mean “parent class.”

It means:

“Go to the next class in MRO.”

So:

`super().method()`

means:

“continue the search chain; don’t restart from the top.”

Crux

The `super()` function does not directly refer to the parent class. Instead, it refers to the next class in the Method Resolution Order. When `super()` is used, Python continues the method search in the inheritance chain without restarting from the top. This allows controlled access to parent class methods.

11. Why method overloading does NOT exist in Python

Because Python does NOT use:

- data types
- function signatures

It only uses:

function name

So:

```
def add(x):  
def add(x, y):
```

The second one replaces the first.

Why?

Because Python decides everything at runtime, not compile time.

Crux

Method overloading does not exist in Python because Python does not differentiate functions based on the number or type of parameters. If multiple methods with the same

name are defined, the last one overrides the previous ones. Since Python resolves everything at runtime and does not use static type checking, traditional method overloading is not supported.

12. How method inheritance works internally

When you call:

```
obj.method()
```

Python does:

Step 1: Check object itself

Does obj have a method?

Step 2: Check class

Step 3: Check parent classes (MRO)

Step 4: Return function

Step 5: Automatically inject **self**

So internally:

```
Class.method(obj)
```

Crux

When a method is called on an object, Python first checks whether the method exists in the object itself. If not found, it checks the class of the object. If it is still not found, Python searches parent classes according to the method resolution order. Once the method is found, Python binds it to the object and automatically passes the object as **self**, allowing the method to operate on that specific instance.